

[NLP & Text Mining]

Seq2Seq with Attention

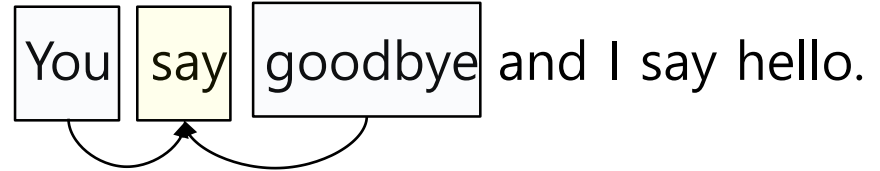
박성호

컴퓨터공학부

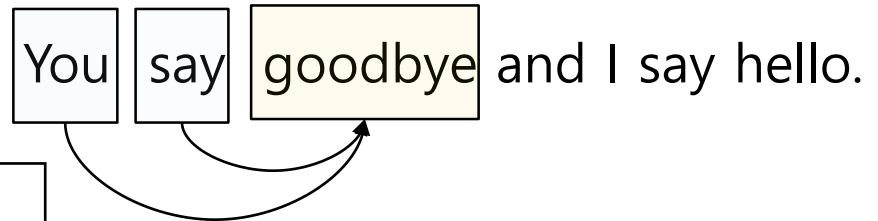
Language Model (언어 모델)

확률 표현

$$p(w_t | w_{t-1}, w_{t+1})$$

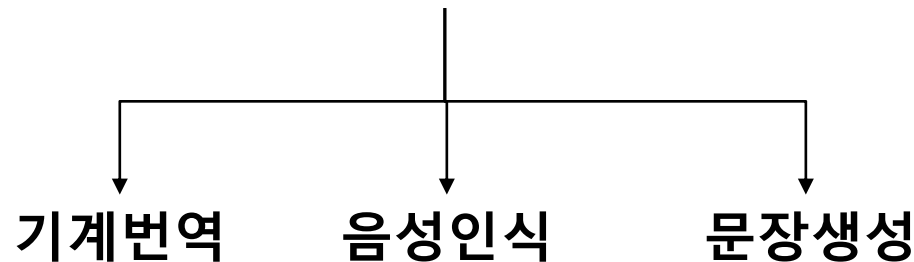


$$p(w_t | w_{t-1}, w_{t-2})$$



= Language Model
(언어 모델)

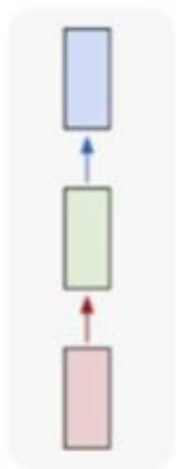
토큰 시퀀스에 확률을 부여하는 모델
이전 단어들이 주어졌을 때 다음 단어를 예측



Language Model (언어 모델) - RNN

RNN = Recurrent Neural Network (Sequence Model)

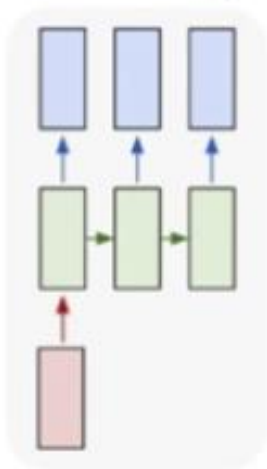
one to one



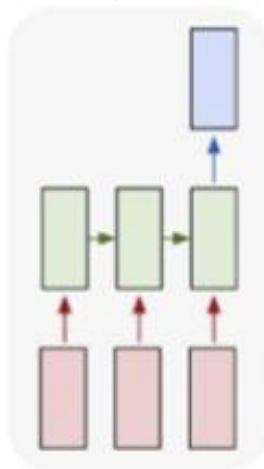
이미지
캡셔닝

input: 이미지
output: 문장

one to many

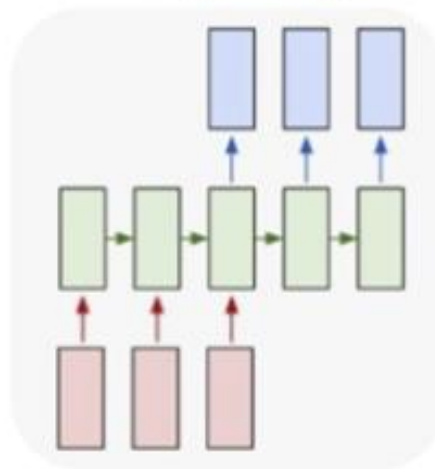


many to one



단어 생성
문장분류
감성 분류

many to many

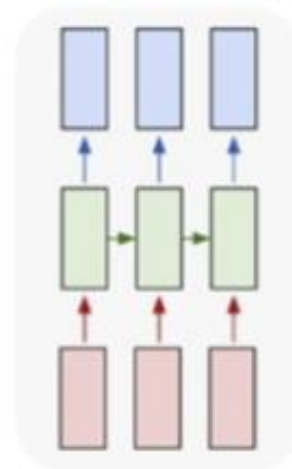


e.g. Machine Translation
seq of words -> seq of words

기계번역

seq-to-seq과 관련
문장을 모두 입력 후
번역

many to many



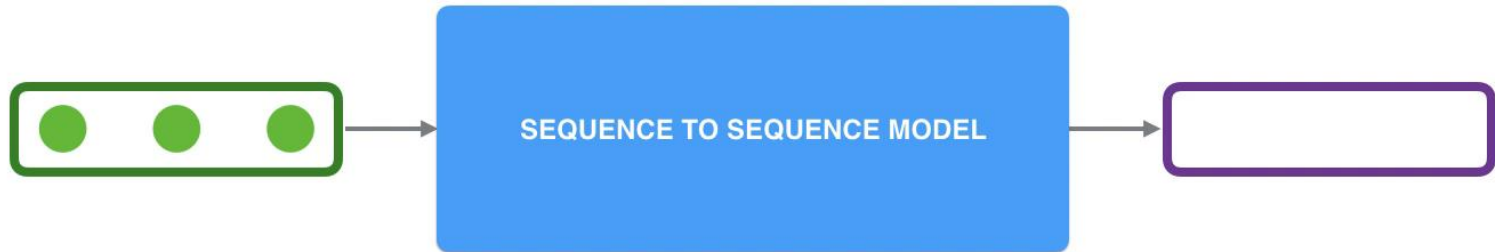
e.g. Machine Translation
seq of words -> seq of words

초기 기계번역
현재는 동영상
데이터 분석

Sequence-to-Sequence (Seq2Seq)

자연어처리 관점에서

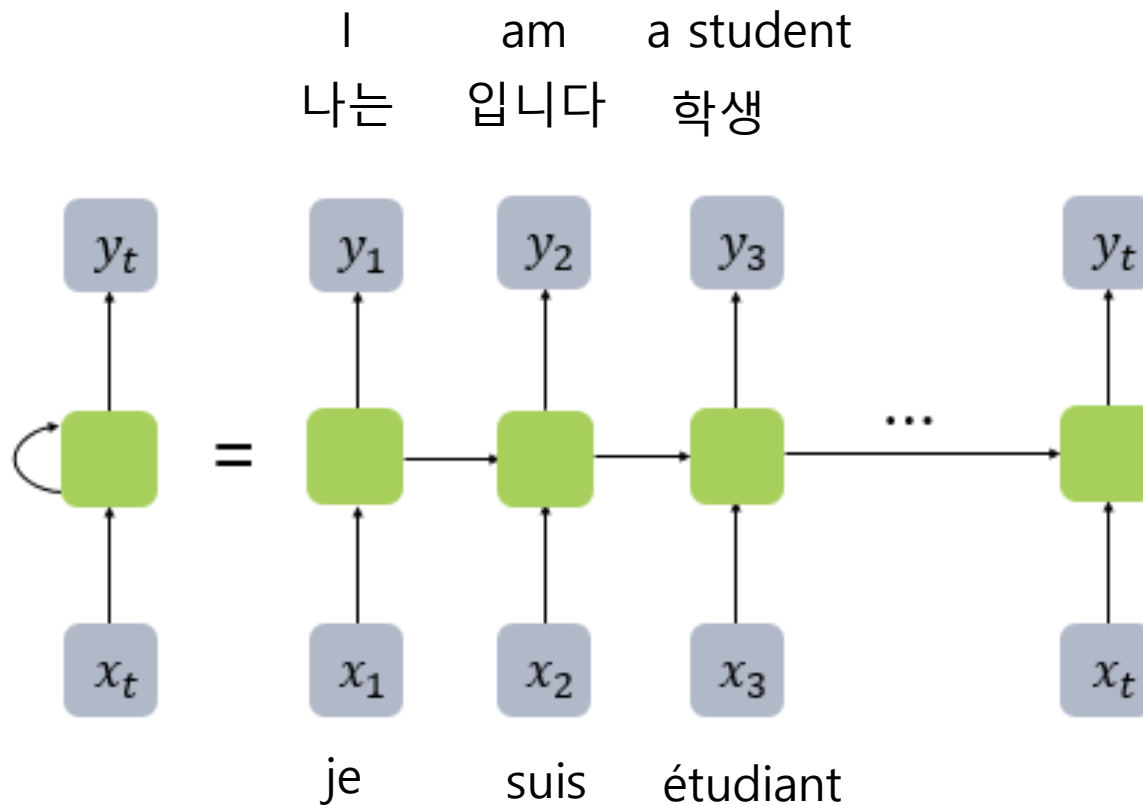
- ❖ 일련의 단어들을 입력 받아 일련의 단어들을 출력함 (Many-to-many)
 - 기계번역 서비스
 - 자동질의응답 서비스
 - 문서요약 서비스
 - ...



Sequence-to-Sequence (Seq2Seq)

기계번역 관점에서

❖ 예시) 불어 문장을 입력 받아 영어 문장을 생성 (번역)



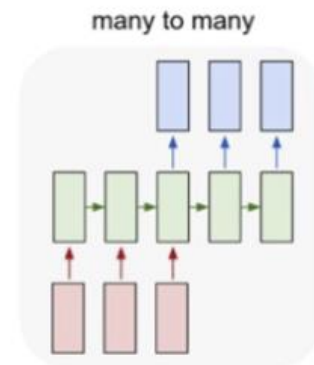
- Input과 Output 간의 길이가 다른 경우
- Input과 Output 간의 의미가 일대일 매칭 안되는 경우

Sequence-to-Sequence (Seq2Seq)

기계번역 관점에서

❖ 예시) 한글 문장을 입력 받아 영어 문장을 생성 (번역)

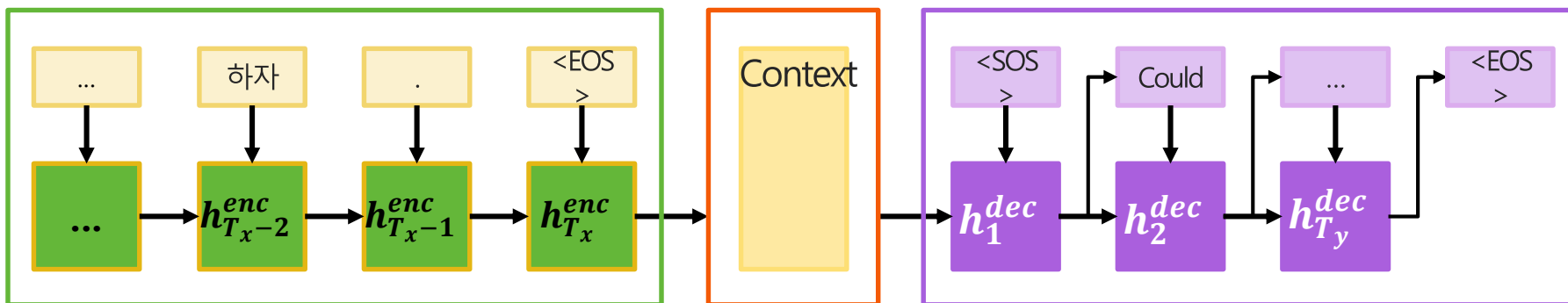
- 불어 문장을 문맥 벡터(Context, ■)로 요약해주는 인코더(Encoder, ■)
- 요약된 문맥 벡터를 활용하여 영어 문장으로 번역하는 디코더(Decoder, ■)
- 마지막에 이 모든 단어 정보들을 압축해서 하나의 벡터로 만드는데, 이를 컨텍스트 벡터(context vector)라고 합니다



(X) 제발 그만 하자.



(Y) Could you please stop it?



- Decoder는 기본적으로 Language model
- 예측 할 때 랑 학습 할 때 랑 다름
- Teacher forcing (교사강요)

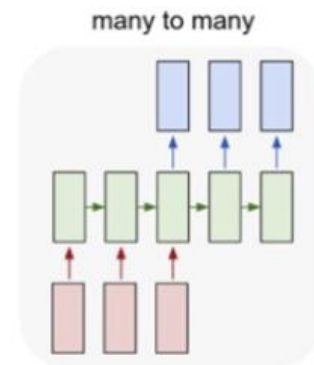
Vanilla RNN이 아니라 **LSTM 셀** 또는 **GRU**

Sequence-to-Sequence (Seq2Seq)

기계번역 관점에서

❖ 예시) 한글 문장을 입력 받아 영어 문장을 생성 (번역)

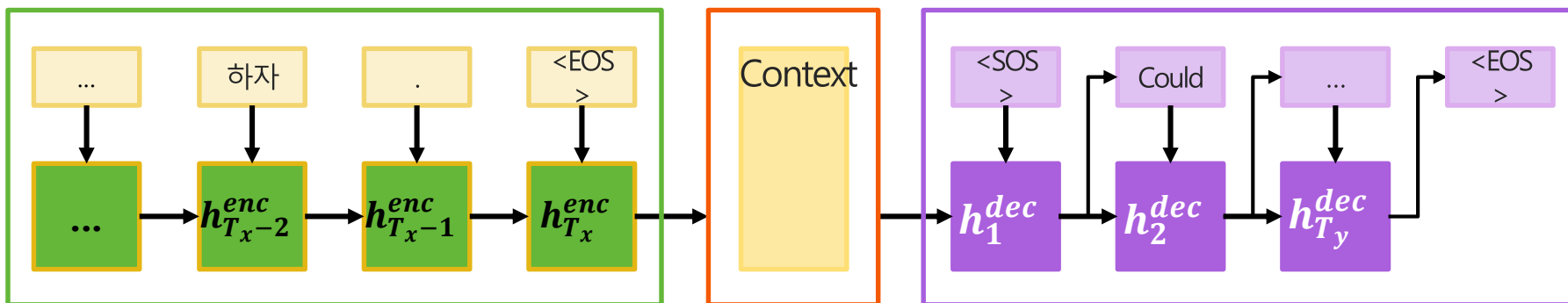
- 불어 문장을 문맥 벡터(Context, ■)로 요약해주는 인코더(Encoder, ■)
- 요약된 문맥 벡터를 활용하여 영어 문장으로 번역하는 디코더(Decoder, ■)
- 마지막에 이 모든 단어 정보들을 압축해서 하나의 벡터로 만드는데, 이를 컨텍스트 벡터(context vector)라고 합니다



(X) 제발 그만 하자.



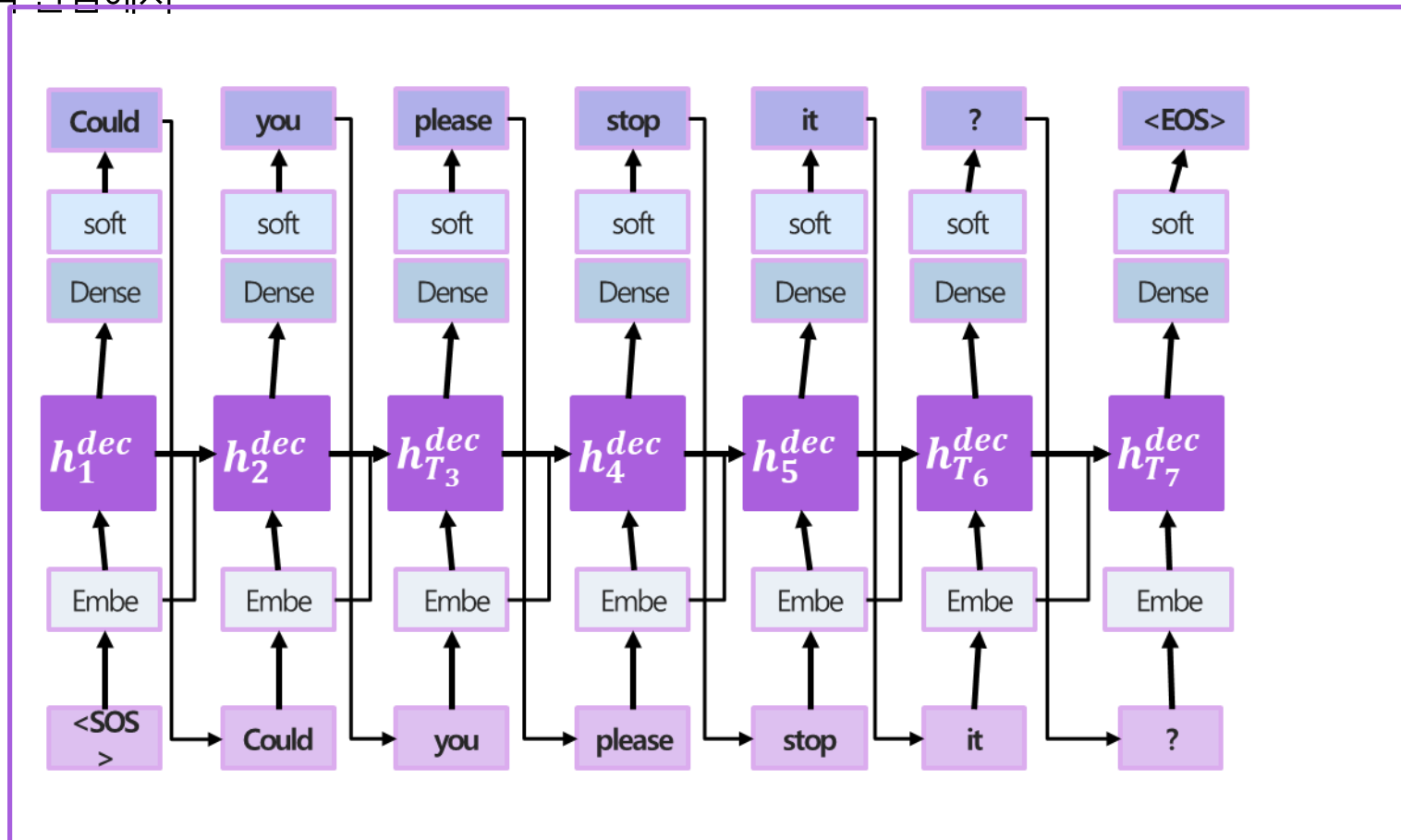
(Y) Could you please stop it?



각 단어는 임베딩(벡터화)된 결과를 사용
Embedding Layer 학습 또는 Pretrained모델 사용

Sequence-to-Sequence (Seq2Seq)

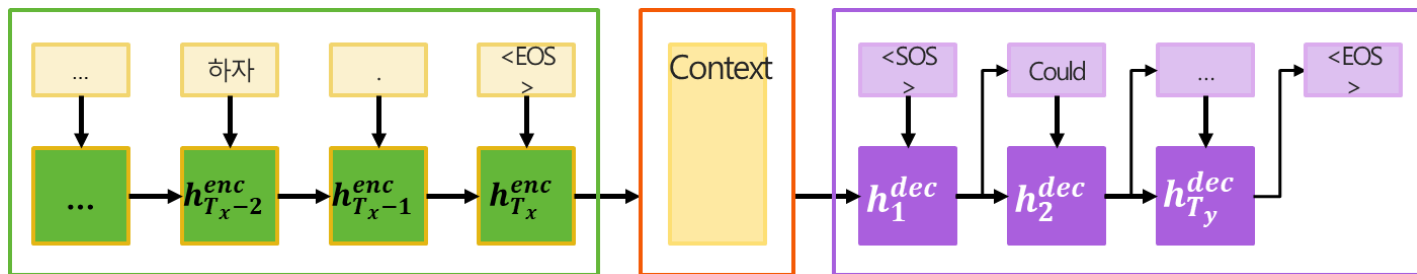
기계번역 관점에서



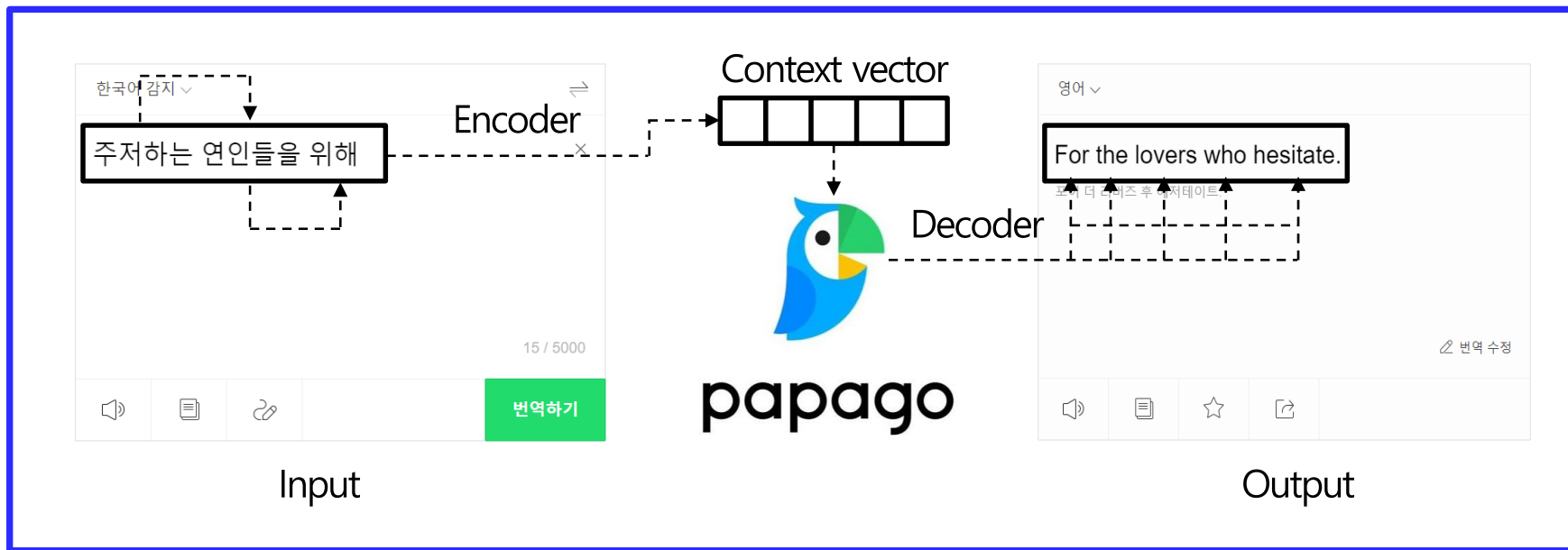
각 단어는 임베딩(벡터화)된 결과를 사용
Embedding Layer 학습 또는 Pretrained모델 사용

Sequence-to-Sequence (Seq2Seq)

기계번역 관점에서



짧은 문장

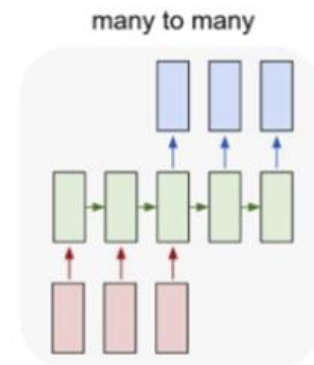


Sequence-to-Sequence (Seq2Seq)

기계번역 관점에서

❖ 예시) 한글 문장을 입력 받아 영어 문장을 생성 (번역)

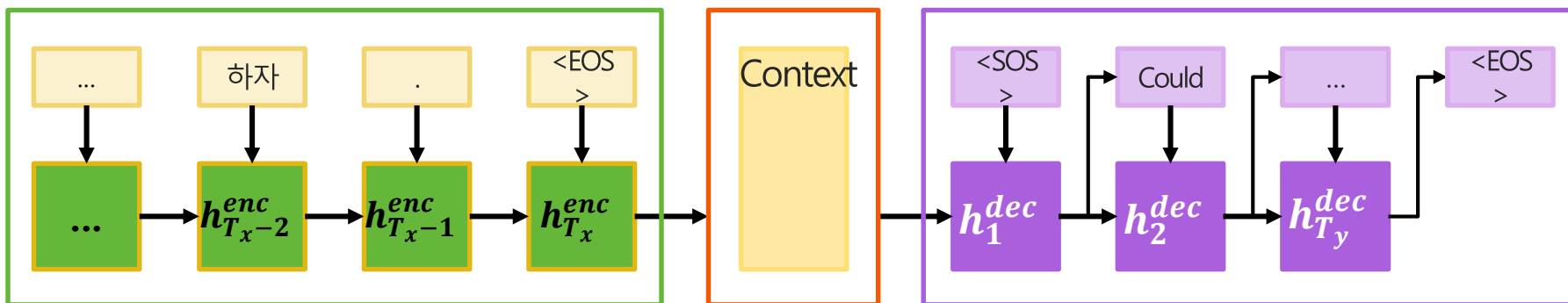
- 불어 문장을 문맥 벡터(Context, ■)로 요약해주는 인코더(Encoder, ■)
- 요약된 문맥 벡터를 활용하여 영어 문장으로 번역하는 디코더(Decoder, ■)
- 마지막에 이 모든 단어 정보들을 압축해서 하나의 벡터로 만드는데, 이를 컨텍스트 벡터(context vector)라고 합니다



(X) 제발 그만 하자.



(Y) Could you please stop it?



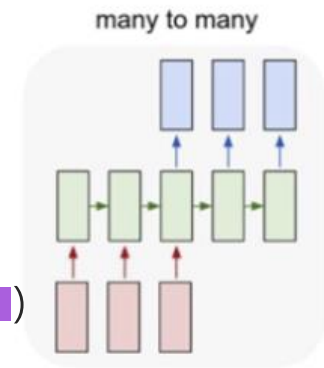
Encoder가 학습한 **context**를
Decoder가 어떻게 활용하느냐가 핵심

What should we use as Context?

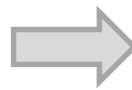
Encoder의 마지막 hidden state 값을 context vector로 사용하자.

❖ 예시) 한글 문장을 입력 받아 영어 문장을 생성 (번역)

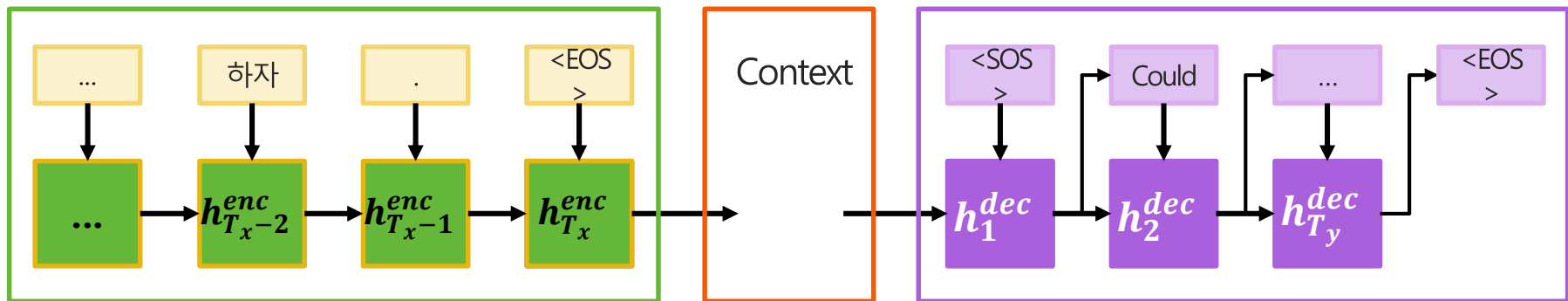
- 불어 문장을 문맥 벡터(Context, ■)로 요약해주는 인코더(Encoder, ■)
- 요약된 문맥 벡터를 활용하여 영어 문장으로 번역하는 디코더(Decoder, ■)



(X) 제발 그만 하자.



(Y) Could you please stop it?



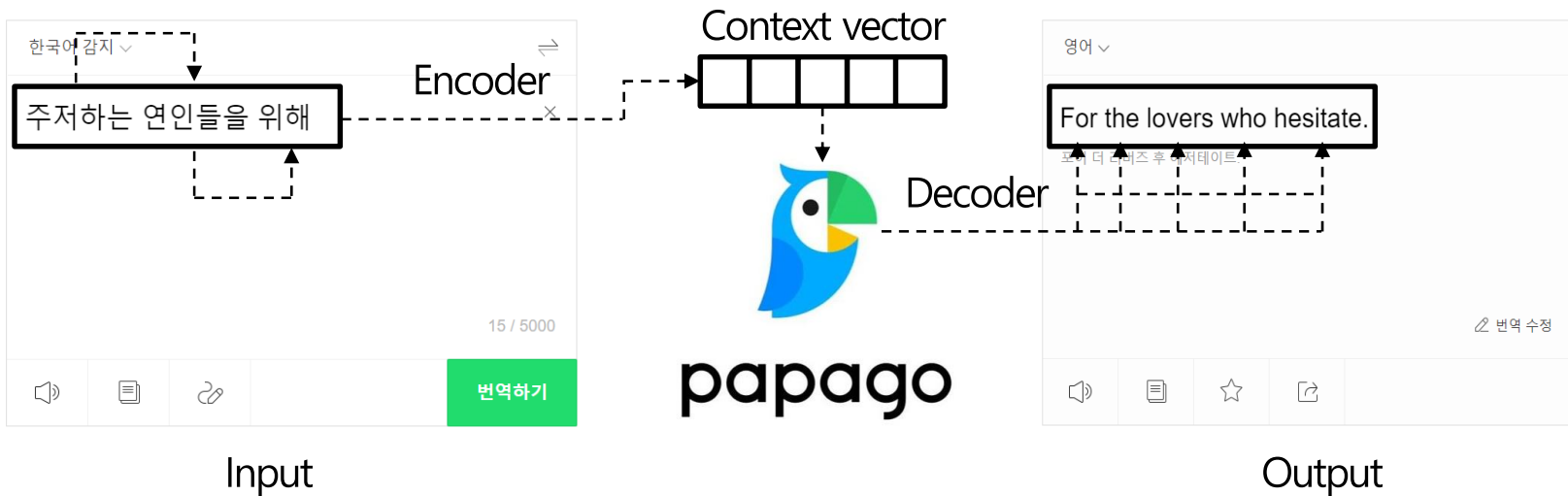
입력 받은 한글 문장을 모두 대변할 수 있을까?

1. 하나의 **고정된 크기의 context vector**에 모든 정보를 압축함으로 **정보 손실이** 발생합니다.
2. RNN의 문제인 기울기 소실(**Vanishing Gradient**)에 따른 학습이 잘 안됨.

Seq2Seq without Attention

왜 필요할까?

- ❖ 짧은 문장은 쉬워요.
- ❖ 굳이 복잡할 모델을 쓸 이유가 없어 보여요.

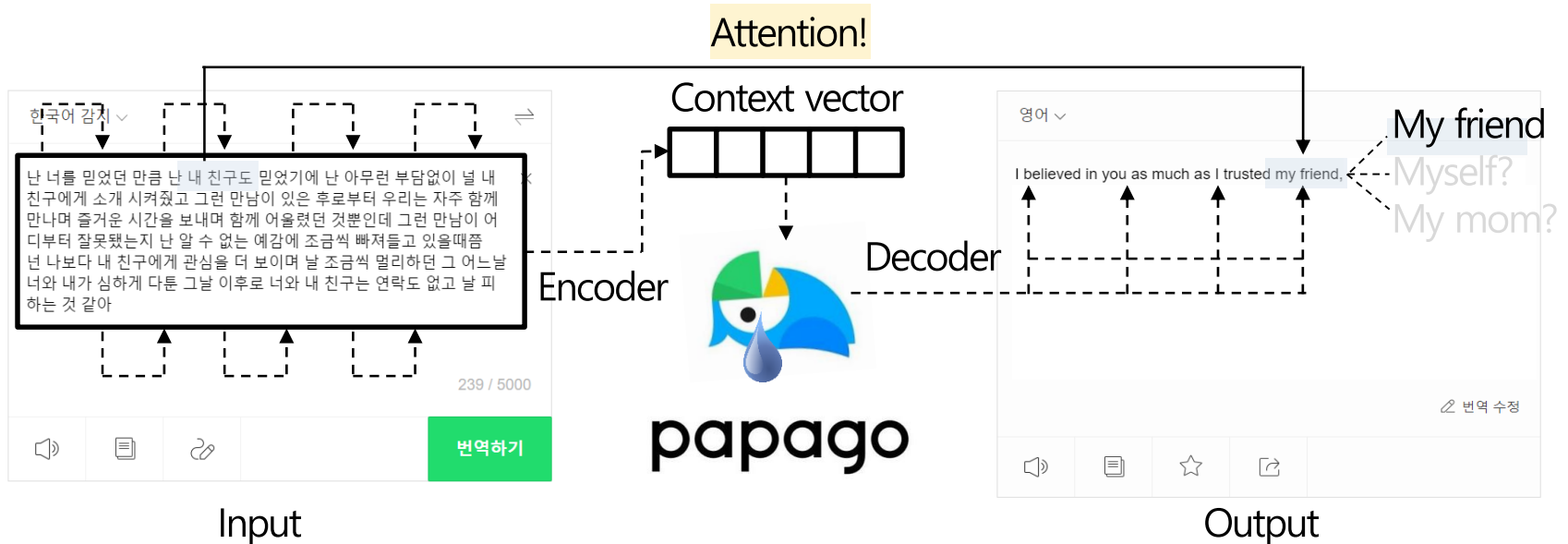


Seq2Seq without Attention

왜 필요할까?

❖ 길어지면 어려워요 :(

- Vanishing Gradient Problem & Long Term Dependency
- 한 글자, 한 글자 써 내려갈 때 나는 도대체 어디에 **집중**해야 할 지 모르겠다...



어디에 집중할 지 어떻게 알지?

What should we use as Context?

마지막 hidden state만 넘겨주지 말고, encoder의 모든 hidden state를 다 넘겨주자!

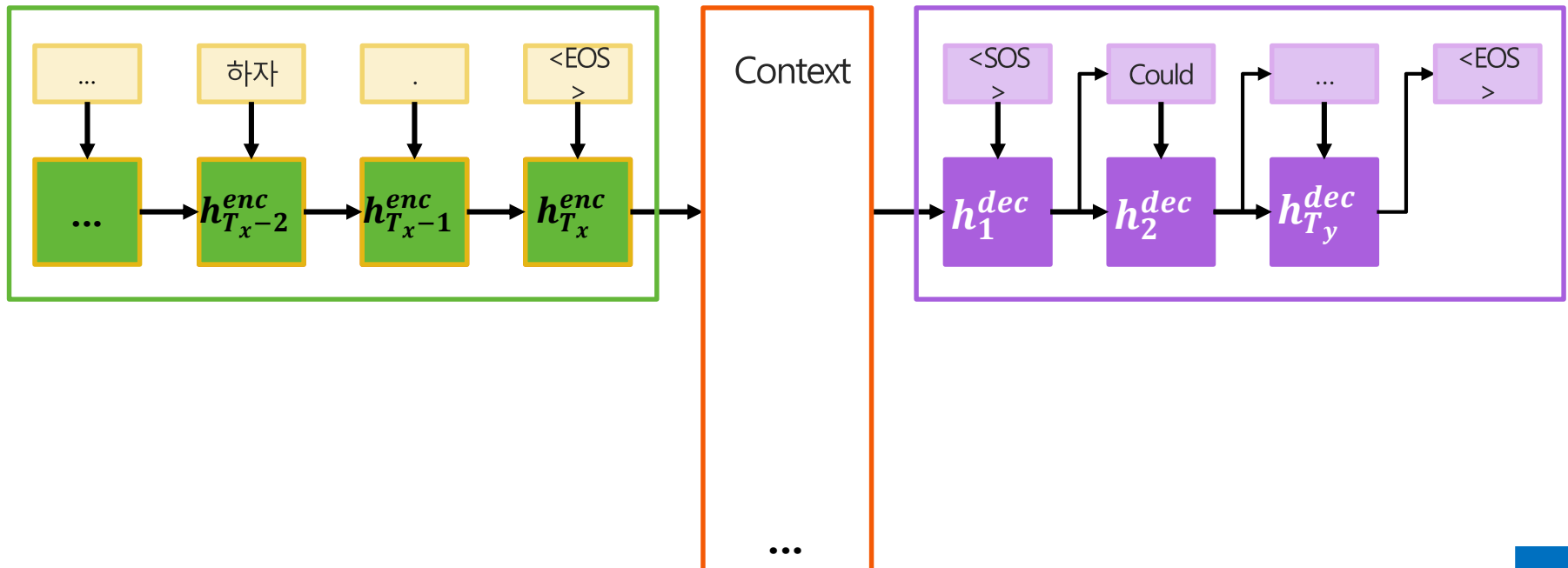
❖ 예시) 한글 문장을 입력 받아 영어 문장을 생성 (번역)

- 불어 문장을 문맥 벡터(Context, ■)로 요약해주는 인코더(Encoder, ■)
- 요약된 문맥 벡터를 활용하여 영어 문장으로 번역하는 디코더(Decoder, ■)

(X) 제발 그만 하자.



(Y) Could you please stop it?



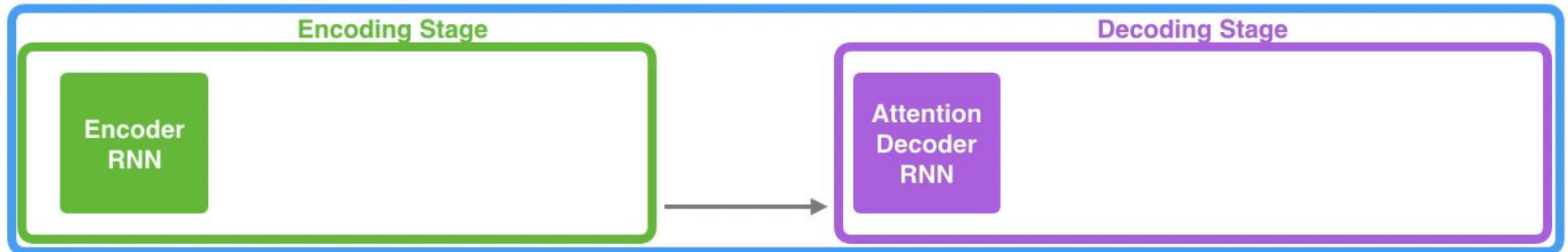
What should we use as Context?

동영상을 보면...

- ❖ 예시) 한글 문장을 입력 받아 영어 문장을 생성 (번역)
 - 불어 문장을 문맥 벡터(Context, ■)로 요약해주는 인코더(Encoder, ■)
 - 요약된 문맥 벡터를 활용하여 영어 문장으로 번역하는 디코더(Decoder, ■)

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Je

suis

étudiant

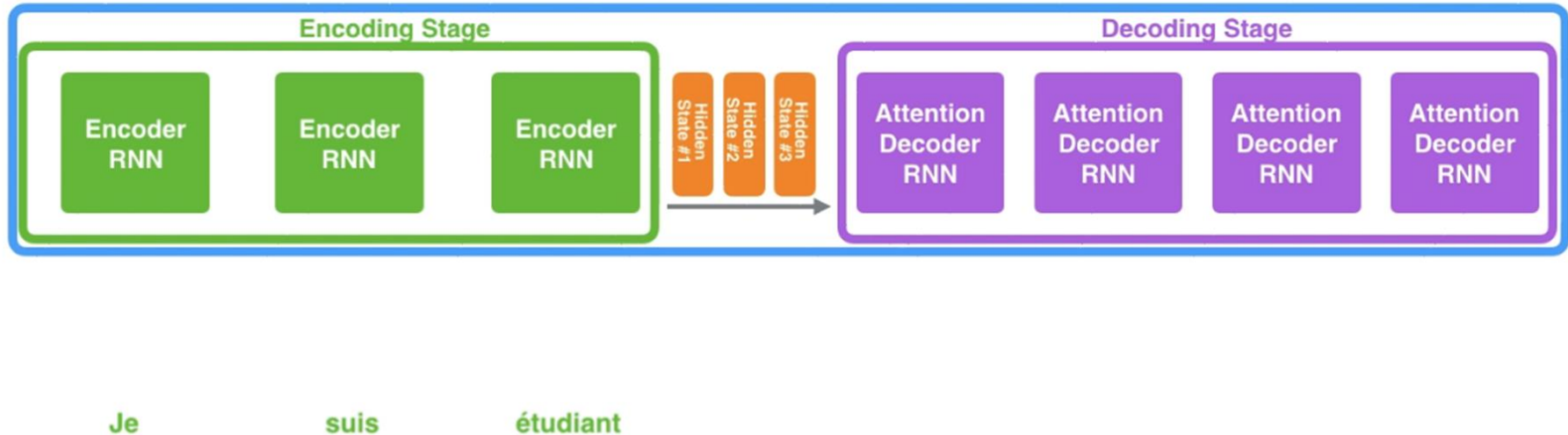
What should we use as Context?

멈춰놓고 보면...

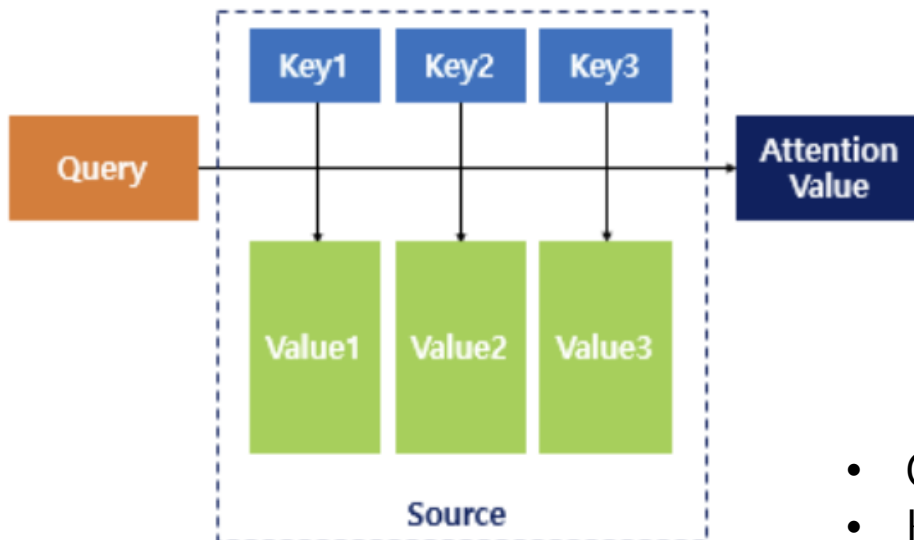
- ❖ 예시) 한글 문장을 입력 받아 영어 문장을 생성 (번역)
 - 불어 문장을 문맥 벡터(Context, ■)로 요약해주는 인코더(Encoder, ■)
 - 요약된 문맥 벡터를 활용하여 영어 문장으로 번역하는 디코더(Decoder, ■)

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Attention Mechanism



1. Query에 대해서 모든 Key와의 유사도 (Attention Score)
2. Attention Score 기반으로 가중치 계산.
3. 가중치 기반으로 Key에 매핑된 value 값에 반영하여 최종 Output 산출

- Query: t시점 Decoder hidden 값
- Keys: 모든 시점의 Encoder hidden 값
- values: 모든 시점의 Encoder hidden 값

I am a student

Neural Machine Translation

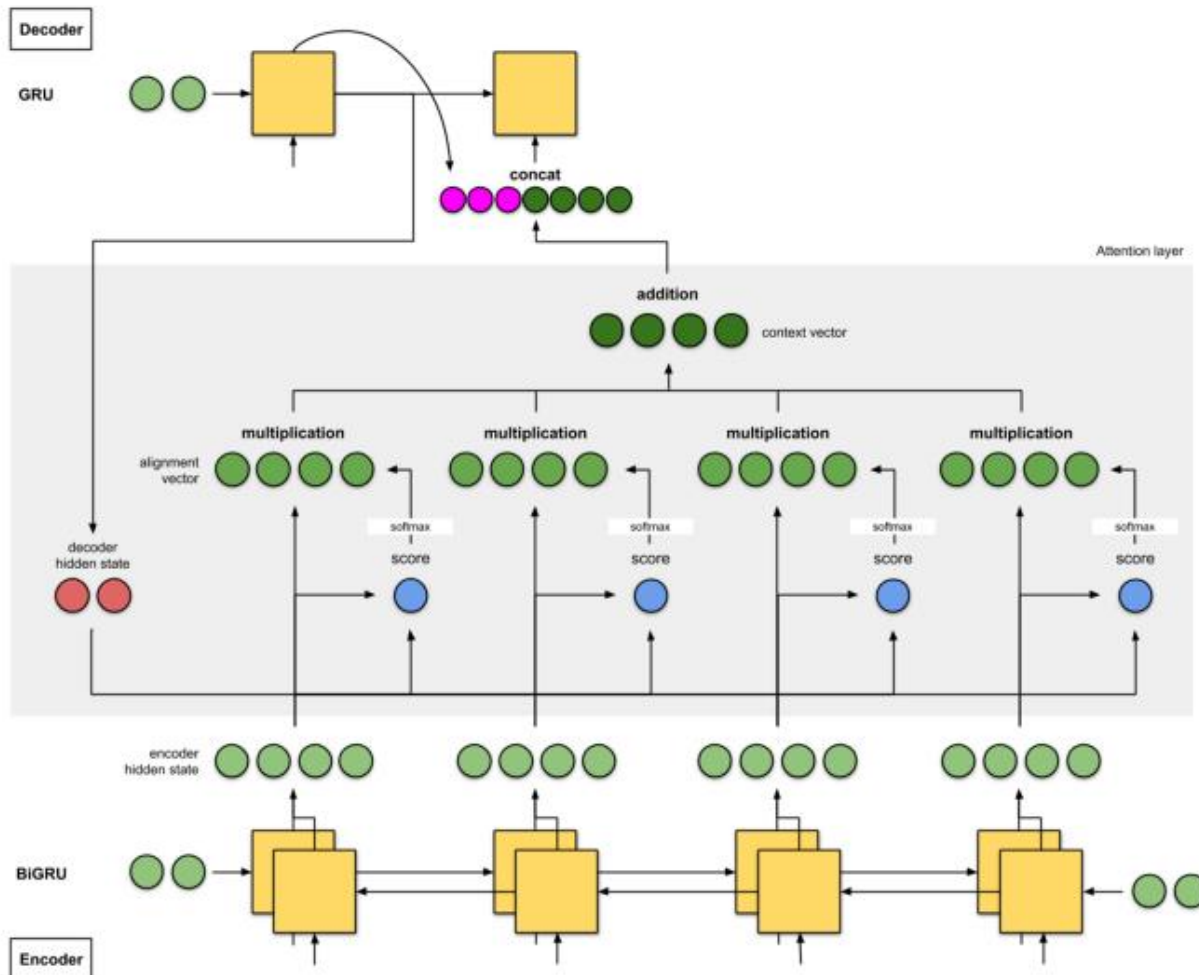
SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Je suis étudiant

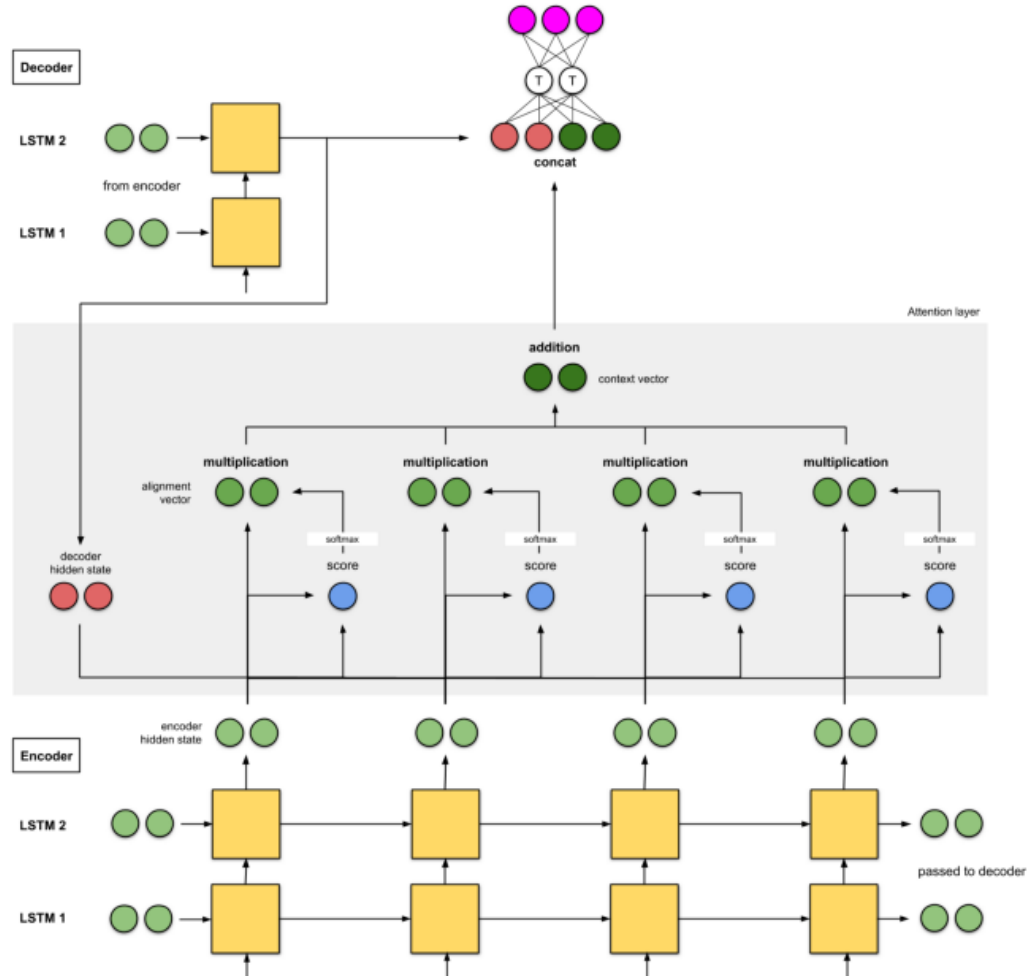
Bahdanau Attention

Bahdanau et al., Neural Machine Translation by Jointly Learning to Align and Translate, 2014



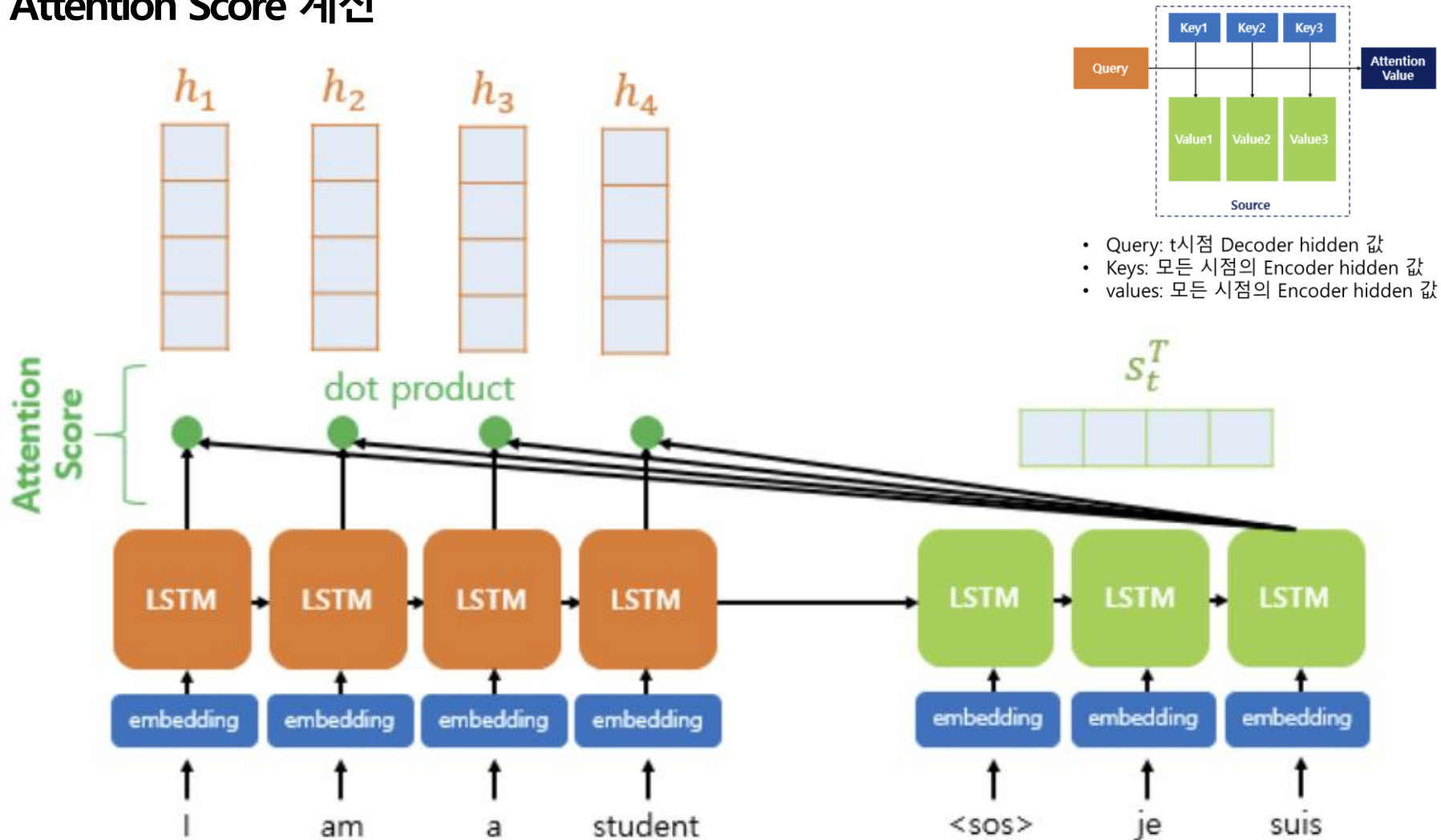
Luong Attention

Luong et al., Effective Approaches to Attention-based Neural Machine Translation, 2015



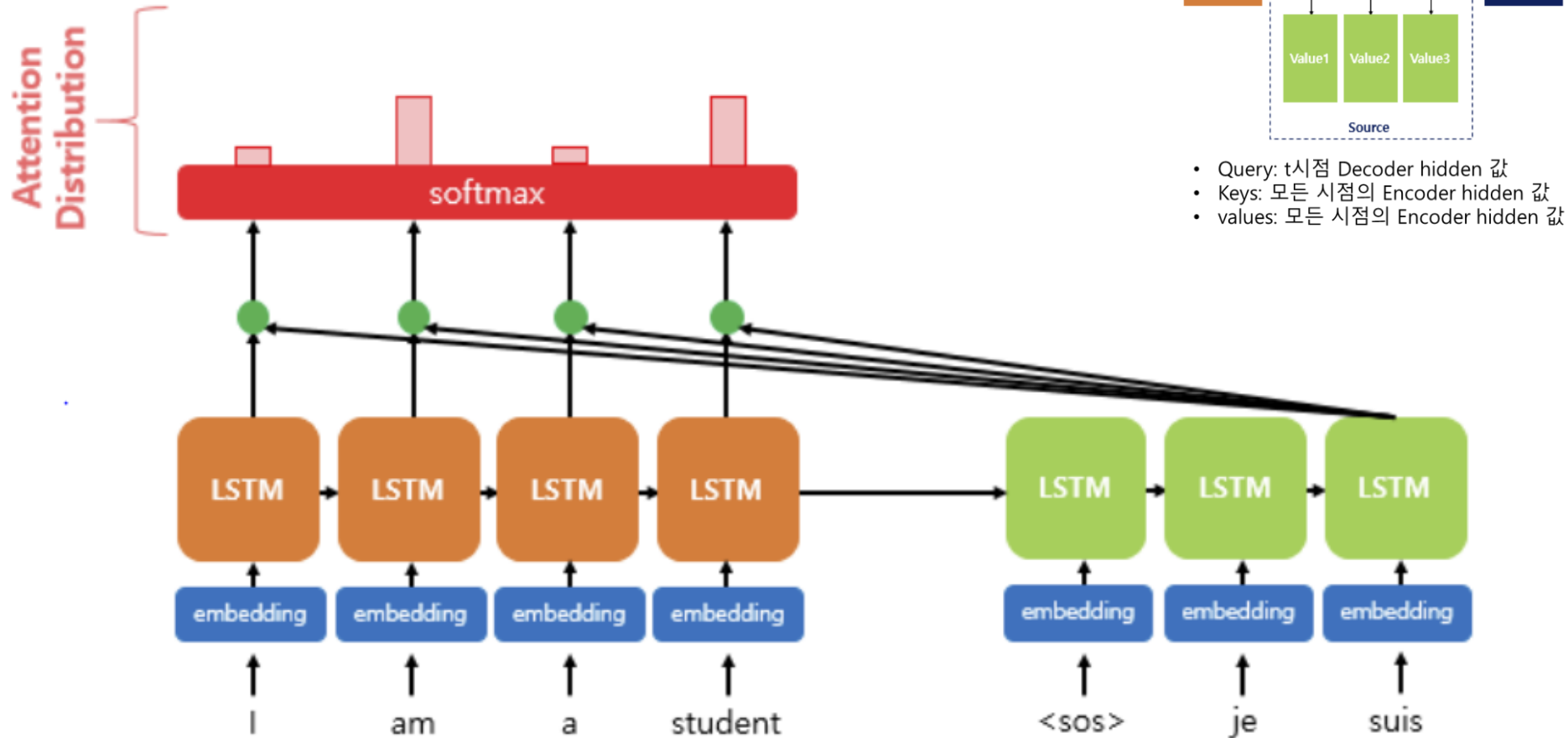
Luong Attention

1. Attention Score 계산



Luong Attention

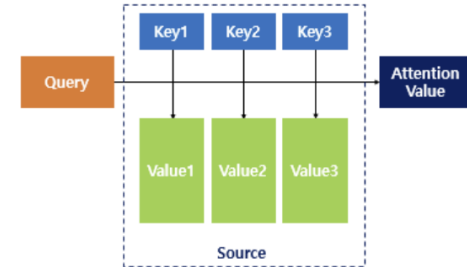
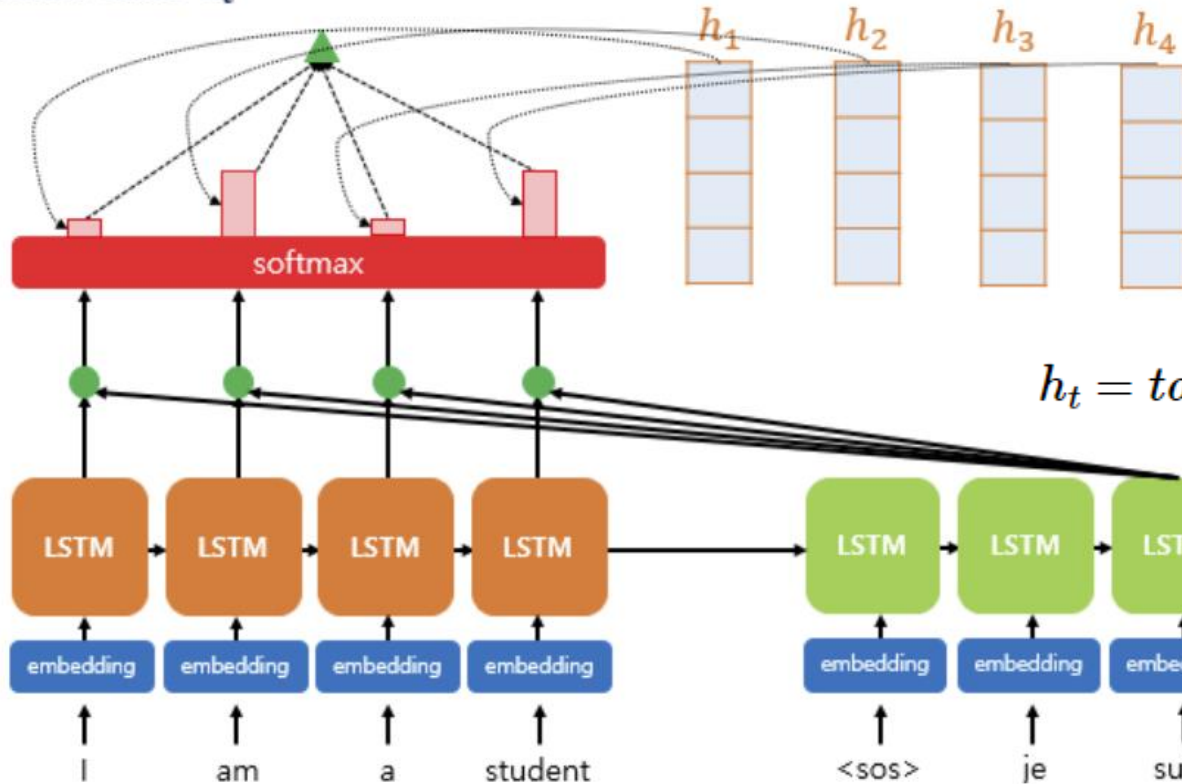
2. Attention 분포 (=가중치 분포) 계산



Luong Attention

3. Attention 가중치를 활용해서 최종 Attention value 계산

Attention Value a_t

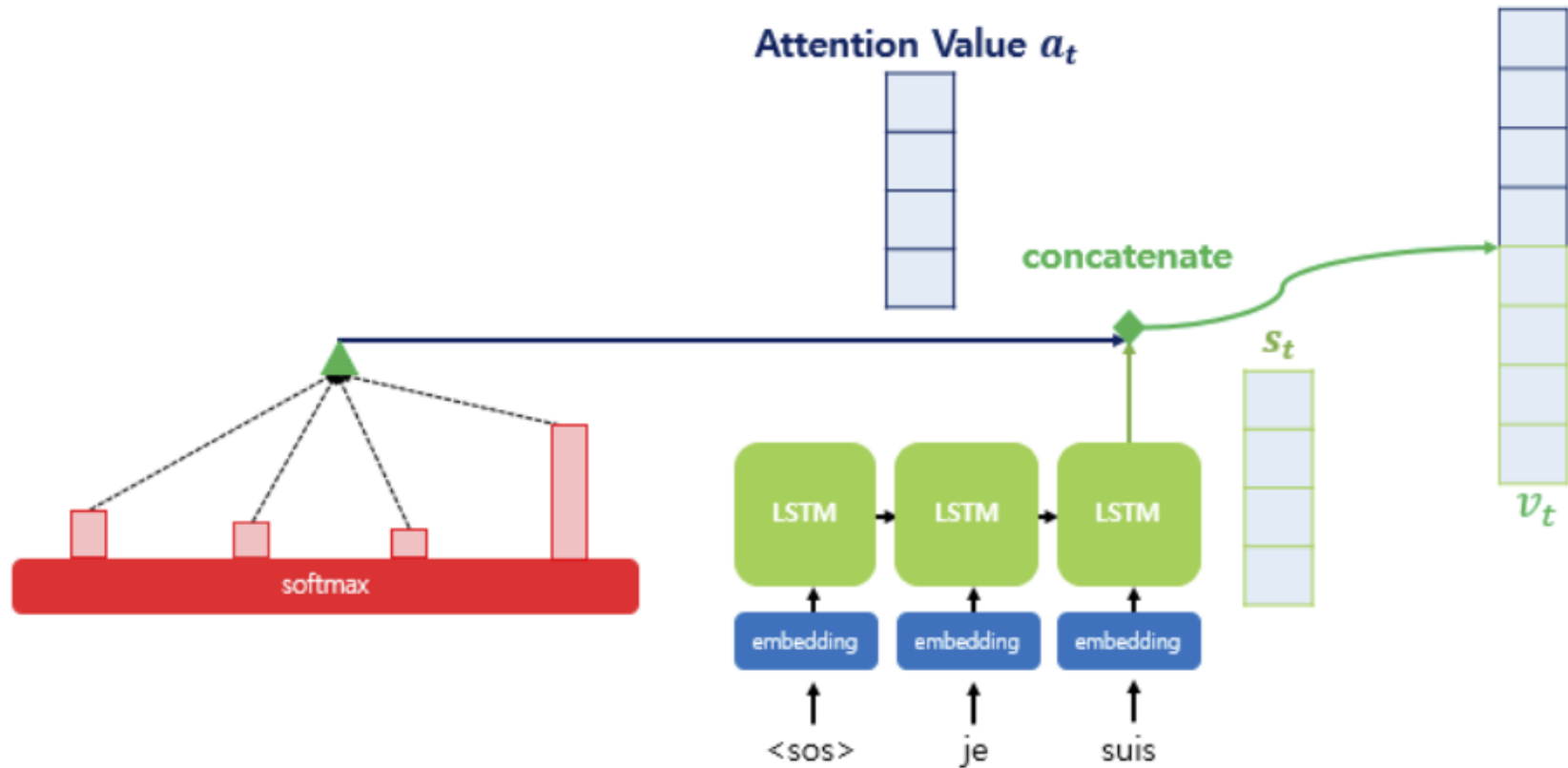


- Query: t시점 Decoder hidden 값
- Keys: 모든 시점의 Encoder hidden 값
- values: 모든 시점의 Encoder hidden 값

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

Luong Attention

4. Hidden 값과 Attention value를 붙인다.



$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

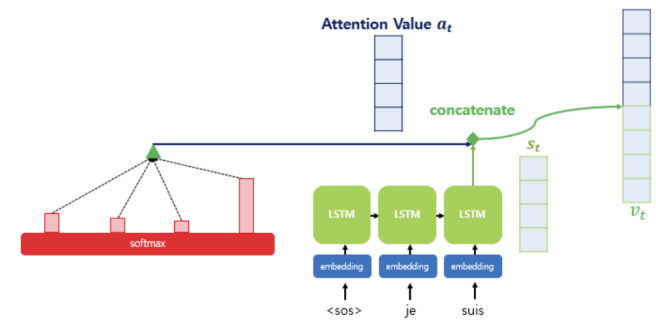
Luong Attention

5. 최종 단어 예측

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Je suis étudiant



Attention Mechanism

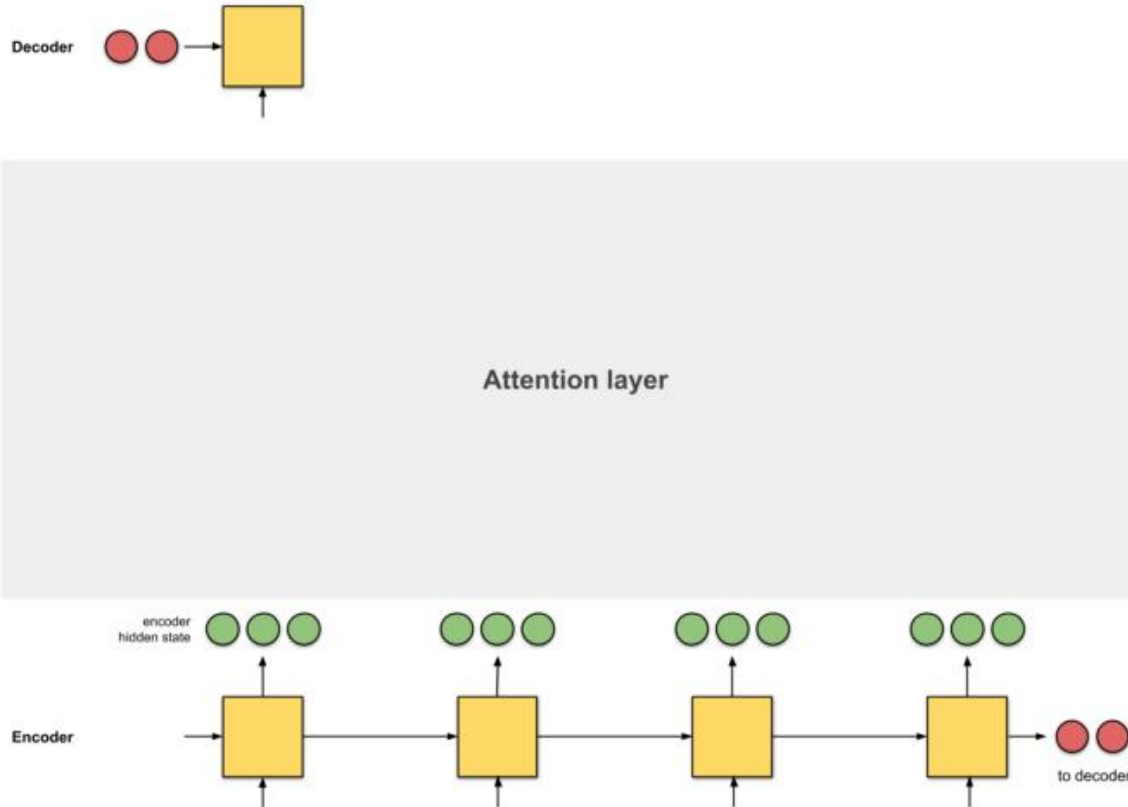
Overall picture

Encoder

<https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3/>

Attention Mechanism

Step 1) Encoder의 마지막 hidden state(● ●)는 Decoder의 initial hidden state(h_0^{dec})로 사용



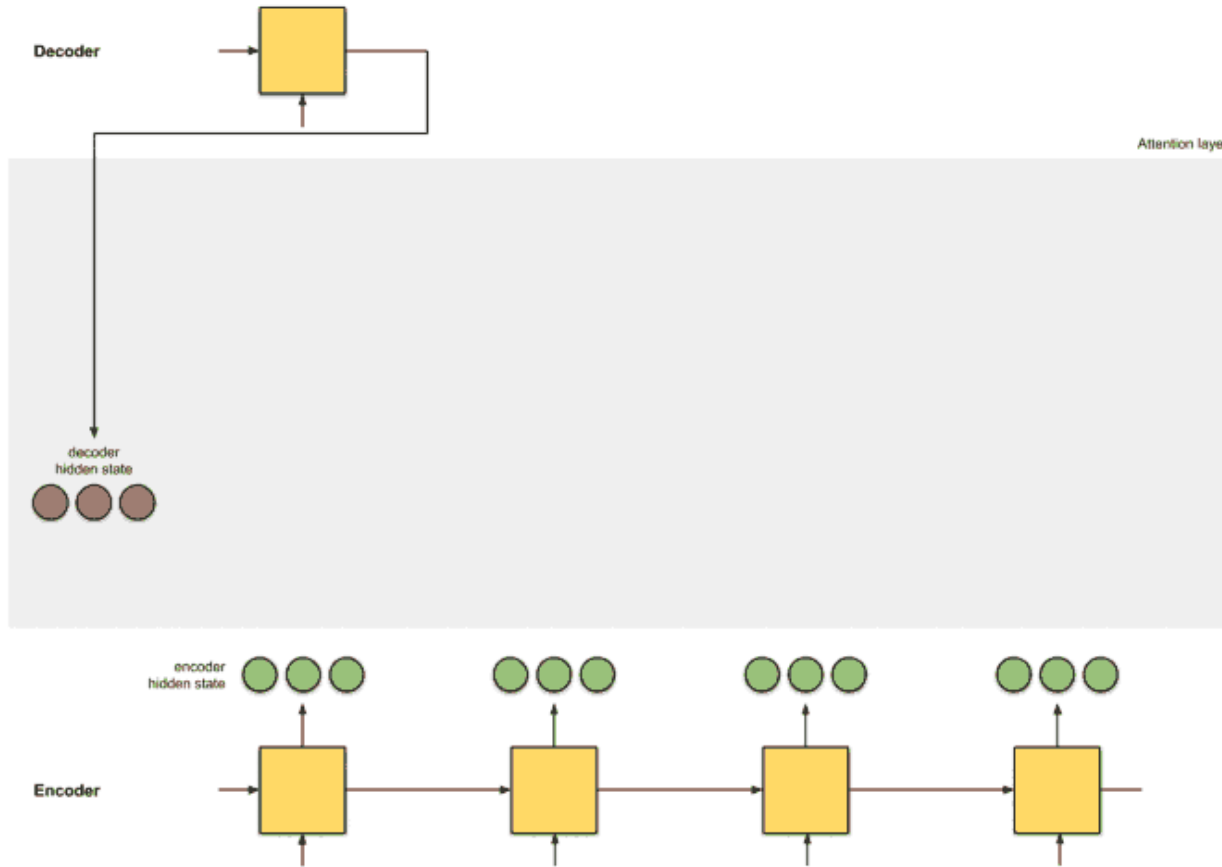
Attention Mechanism

Step 2) Decoder의 새로운 hidden state(h_1^{dec} , ●) 값 계산

Encoder

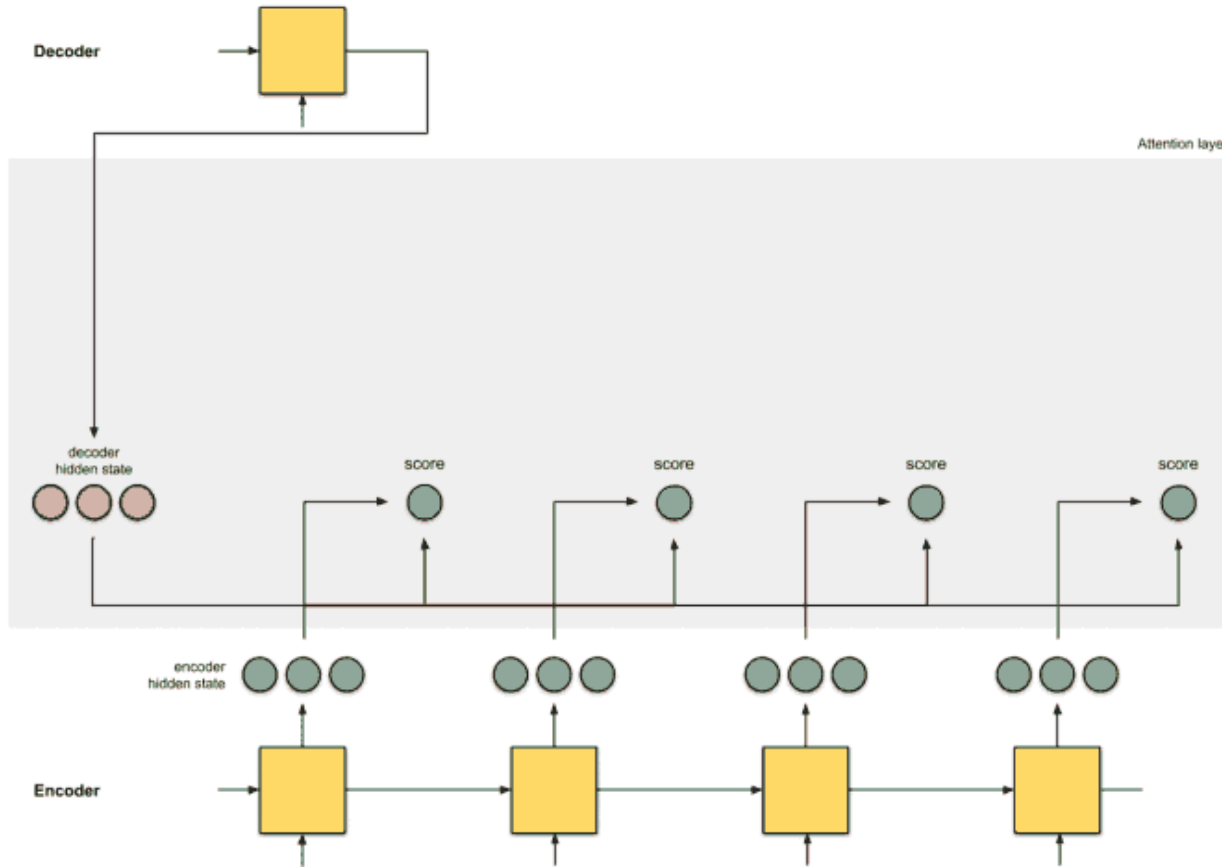
Attention Mechanism

Step 3) Decoder의 새로운 hidden state (h_1^{dec} , ●)와 encoder hidden states ($h_{1:T_x}^{enc}$, ●)로 $\text{score}(\bullet)$ 를 계산



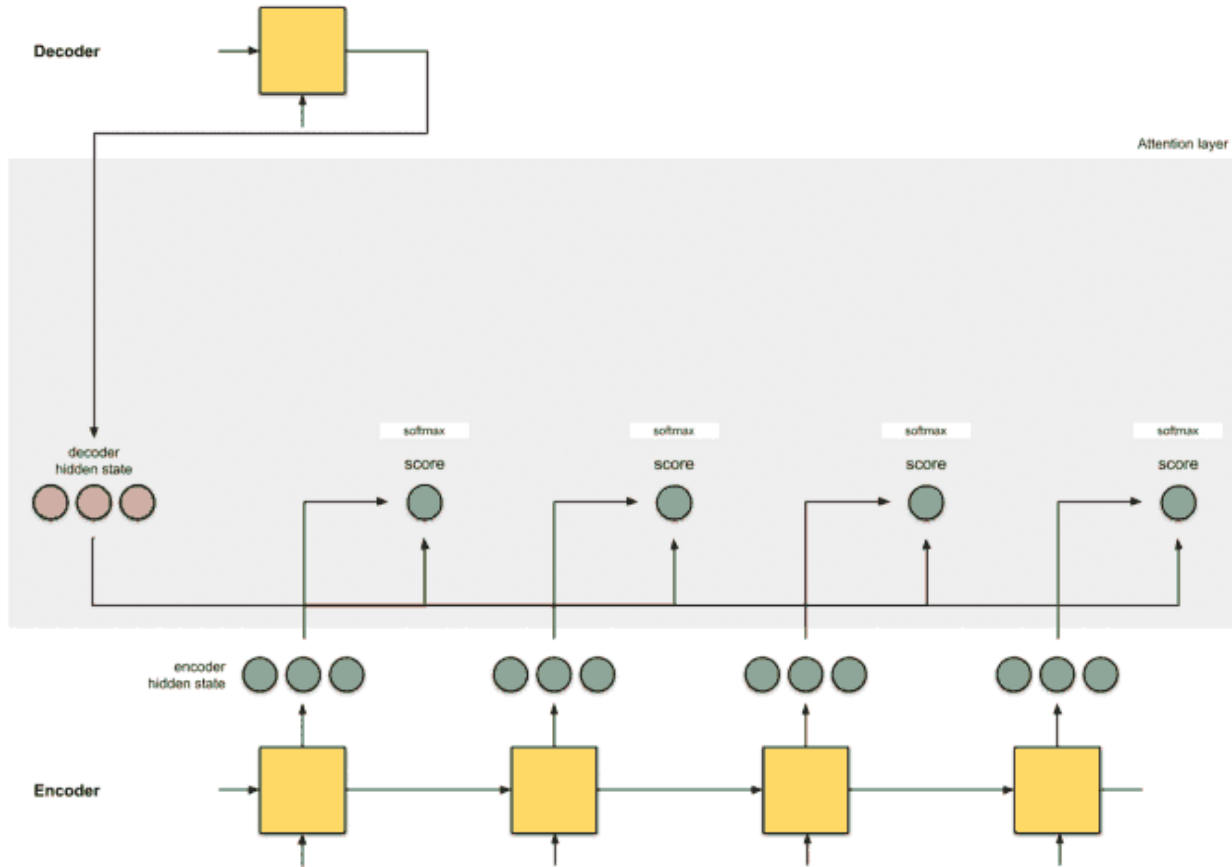
Attention Mechanism

Step 4) Softmax 함수를 적용하여 점수들의 총합이 1이 되도록 맞춰주기



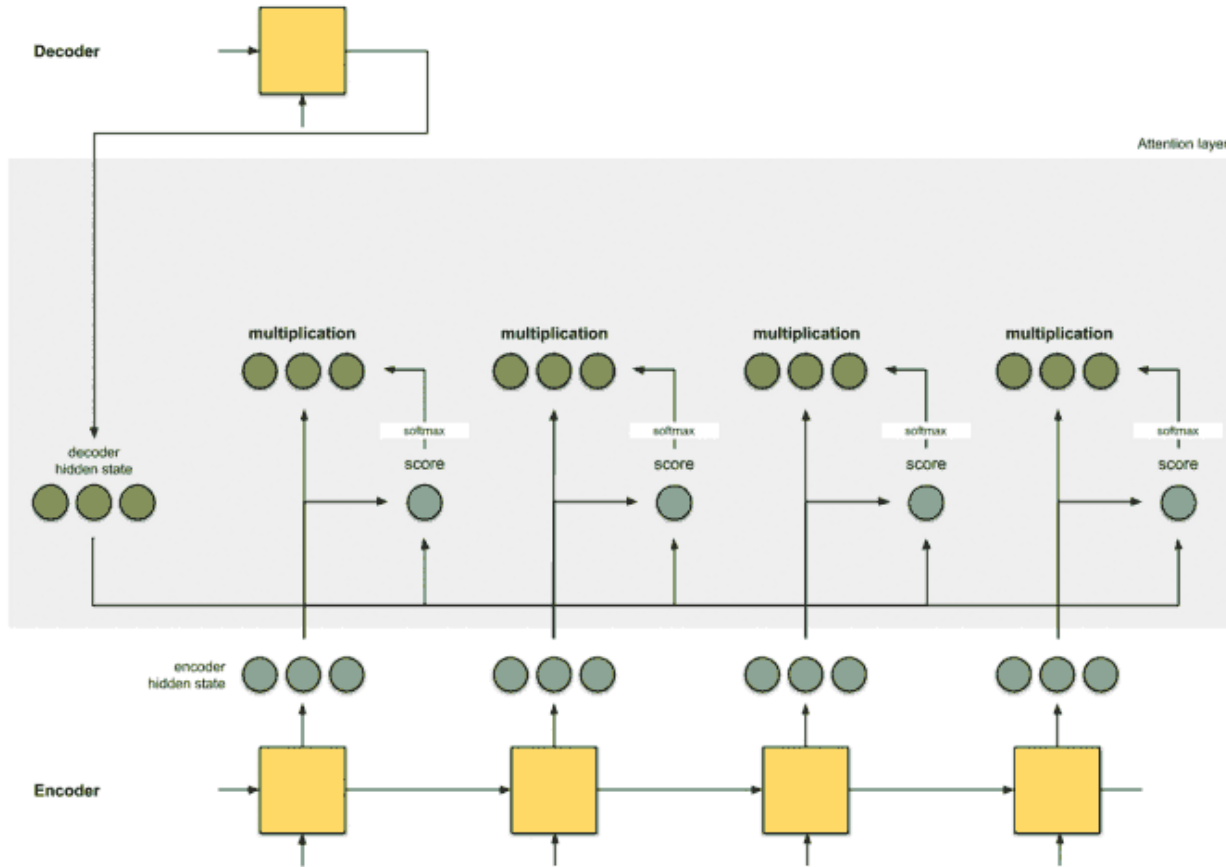
Attention Mechanism

Step 5) 각 encoder 시점($1:T_x$)의 hidden state(h_t^{enc}) 값과 score를 곱하기



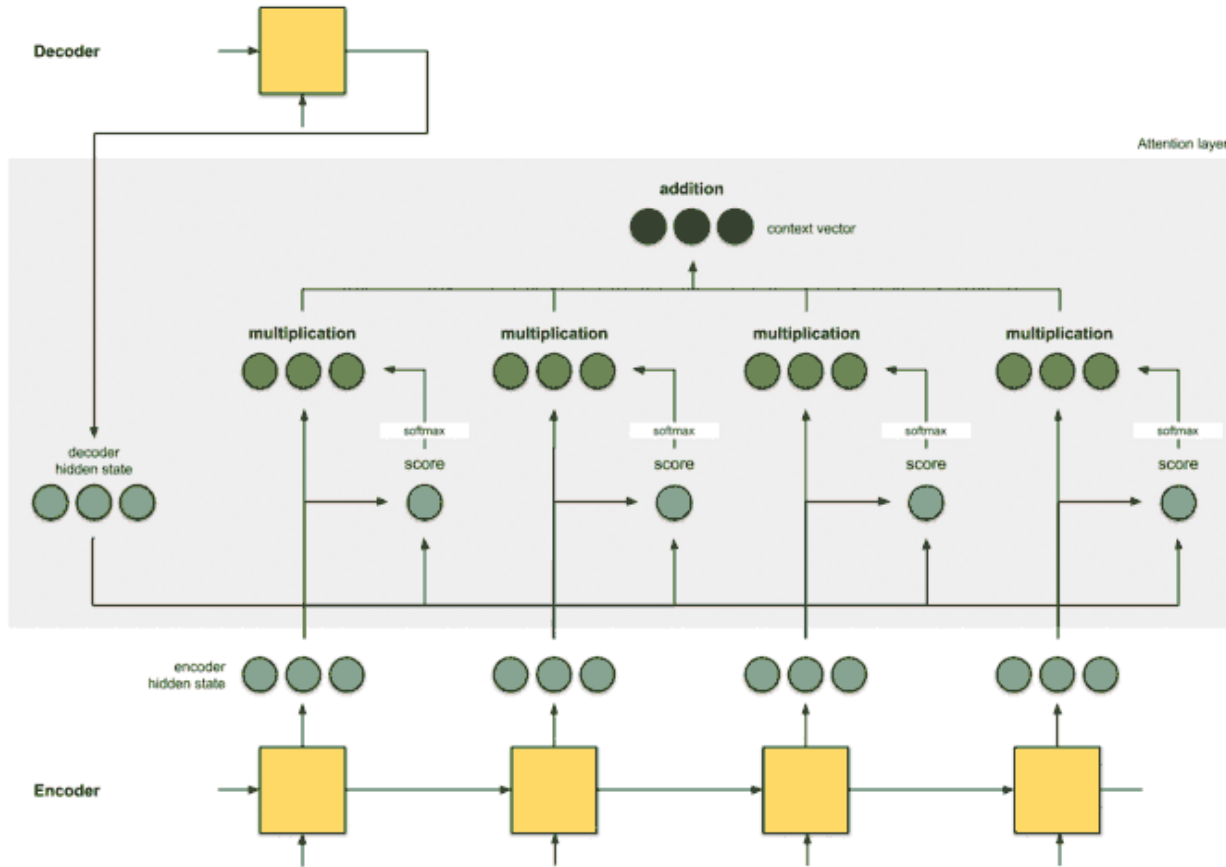
Attention Mechanism

Step 6) 각 encoder 시점($1:T_x$)의 hidden state(h_t^{enc}) 값과 score를 곱하여 합하기 → 가중평균(●)...



Attention Mechanism

Step 7) Context vector(●) 값과 현 시점의 최종 예측 벡터를 옆으로 붙이고 다음 시점으로 진행

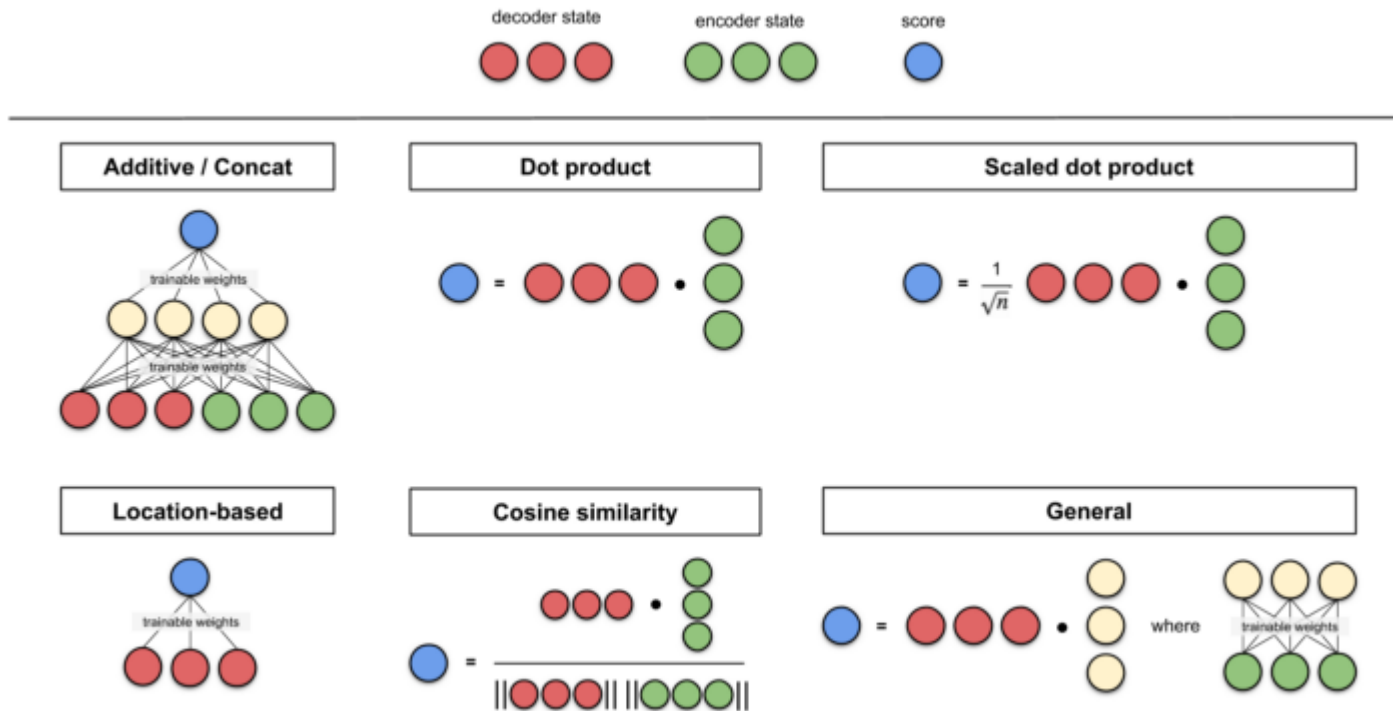


Attention Score Functions

'유사도를 계산하는 다양한 방식이 존재한다' 정도로 이해하면 됩니다.



(Encoder hidden states($h_{1:T_x}^{enc}$)와 decoder hidden state(h_t^{dec}) 사이의)



Attention Score Functions

'유사도를 계산하는 다양한 방식이 존재한다' 정도로 이해하면 됩니다.



(Encoder hidden states($h_{1:T_x}^{enc}$)와 decoder hidden state(h_t^{dec}) 사이의)

$$\alpha_{ts} = \frac{\exp(\text{score}(\mathbf{h}_t, \bar{\mathbf{h}}_s))}{\sum_{s'=1}^S \exp(\text{score}(\mathbf{h}_t, \bar{\mathbf{h}}_{s'}))} \quad [\text{Attention weights}] \quad (1)$$

$$\mathbf{c}_t = \sum_s \alpha_{ts} \bar{\mathbf{h}}_s \quad [\text{Context vector}] \quad (2)$$

$$\mathbf{a}_t = f(\mathbf{c}_t, \mathbf{h}_t) = \tanh(\mathbf{W}_c[\mathbf{c}_t; \mathbf{h}_t]) \quad [\text{Attention vector}] \quad (3)$$

$$\text{score}(\mathbf{h}_t, \bar{\mathbf{h}}_s) = \begin{cases} \mathbf{h}_t^\top \mathbf{W} \bar{\mathbf{h}}_s & [\text{Luong's multiplicative style}] \\ \mathbf{v}_a^\top \tanh(\mathbf{W}_1 \mathbf{h}_t + \mathbf{W}_2 \bar{\mathbf{h}}_s) & [\text{Bahdanau's additive style}] \end{cases} \quad (4)$$

Attention Score Functions

'유사도를 계산하는 다양한 방식이 존재한다' 정도로 이해하면 됩니다.

Name	Alignment score function	Citation
Content-base attention	$\text{score}(s_t, h_i) = \text{cosine}[s_t, h_i]$	Graves2014
Additive(*)	$\text{score}(s_t, h_i) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[s_t; h_i])$	Bahdanau2015
Location-Base	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a s_t)$ Note: This simplifies the softmax alignment to only depend on the target position.	Luong2015
General	$\text{score}(s_t, h_i) = s_t^\top \mathbf{W}_a h_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer.	Luong2015
Dot-Product	$\text{score}(s_t, h_i) = s_t^\top h_i$	Luong2015
Scaled Dot-Product(^)	$\text{score}(s_t, h_i) = \frac{s_t^\top h_i}{\sqrt{n}}$ Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.	Vaswani2017

Loung (Multiplicative Attention)

수식과 단계

Luong Attention에는 세 가지 변형이 있습니다: **Dot Product**, **General**, **Concat**.

여기서는 가장 일반적인 **Dot Product** 방식을 설명합니다.

1. Alignment Score 계산 (점곱 방식):

Query (s_j)와 Key (h_i)의 점곱을 통해 Alignment Score를 계산합니다.

$$e_{ij} = h_i^T s_j$$

- e_{ij} : Alignment Score (디코더 현재 hidden state와 인코더 hidden state 간의 점곱 결과)
- h_i : 인코더의 i -번째 hidden state (Key)
- s_j : 디코더의 현재 hidden state (Query)

2. Attention 가중치 계산:

Alignment Score e_{ij} 에 softmax를 적용해 Attention 가중치 α_{ij} 를 구합니다.

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_k \exp(e_{kj})}$$

3. Context Vector 계산:

Attention 가중치 α_{ij} 를 인코더 hidden states h_i 에 가중합합니다.

$$c_j = \sum_i \alpha_{ij} h_i$$

- c_j : Context Vector

4. 디코더 출력 계산:

Context Vector c_j 와 디코더의 현재 hidden state s_j 를 결합하여 최종 출력을 생성합니다.

$$\tilde{s}_j = \tanh(W_c[c_j; s_j])$$

- W_c : 학습 가능한 가중치 행렬
- $\tilde{s}_j$: 디코더의 최종 출력



특징 요약

1. Multiplicative Attention:

Query와 Key의 점곱을 사용해 유사도를 계산합니다.

2. Query: 현재 디코더 hidden state (s_j)

3. Key/Value: 인코더 hidden states (h_i)

4. Attention 적용 시점: 디코더 현재 상태를 사용하여 Attention을 계산합니다.

5. 인코더: 단방향 RNN을 주로 사용합니다.

Bahdanau (Additive Attention)

수식과 단계

1. Alignment Score 계산:

Query (s_{j-1})와 Key (h_i)를 선형 변환한 후 더하고, **tanh 활성화 함수**를 적용합니다.

$$e_{ij} = v_a^T \tanh(W_a h_i + U_a s_{j-1})$$

- e_{ij} : Alignment Score (인코더의 i -번째 hidden state와 디코더의 이전 hidden state 사이의 유사도)
- h_i : 인코더의 i -번째 hidden state (Key)
- s_{j-1} : 디코더의 이전 hidden state (Query)
- W_a, U_a : 학습 가능한 가중치 행렬
- v_a : 학습 가능한 벡터

2. Attention 가중치 계산:

Alignment Score e_{ij} 에 softmax를 적용해 정규화된 Attention 가중치 α_{ij} 를 구합니다.

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_k \exp(e_{kj})}$$

3. Context Vector 계산:

Attention 가중치 α_{ij} 를 인코더의 hidden states h_i 에 가중합합니다.

$$c_j = \sum_i \alpha_{ij} h_i$$

- c_j : Context Vector (현재 시점에서 인코더 정보를 요약한 벡터)

4. 디코더 출력 계산:

Context Vector c_j 와 디코더의 이전 hidden state s_{j-1} 를 결합하여 현재 시점의 hidden state를 계산합니다.

$$s_j = \text{RNN}(y_{j-1}, [s_{j-1}, c_j])$$

- y_{j-1} : 이전 시점의 디코더 출력

특징 요약

1. Additive Attention:

Query와 Key를 선형 변환 후 합친 다음 비선형 활성화 함수를 적용합니다.

2. Query: 이전 디코더 hidden state (s_{j-1})

3. Key/Value: 인코더 hidden states (h_i)

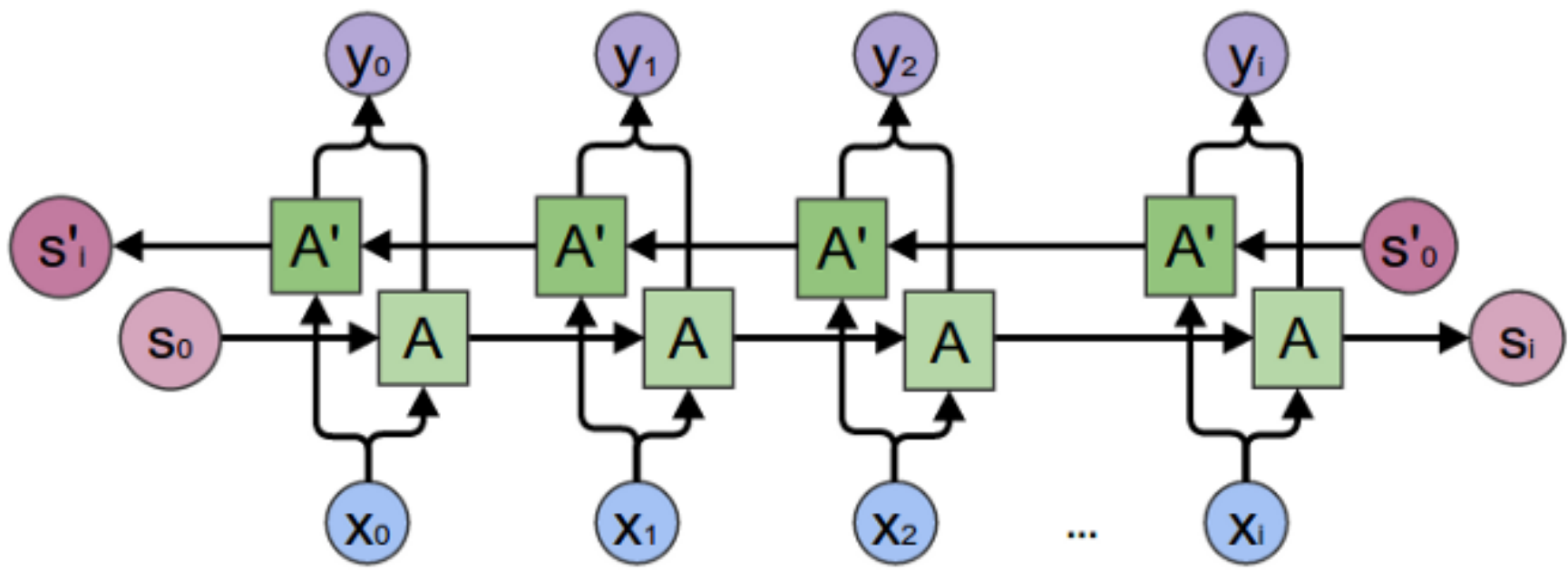
4. Attention 적용 시점: 디코더 이전 상태를 사용하여 Attention을 계산합니다.

5. 인코더: Bidirectional RNN을 주로 사용합니다.

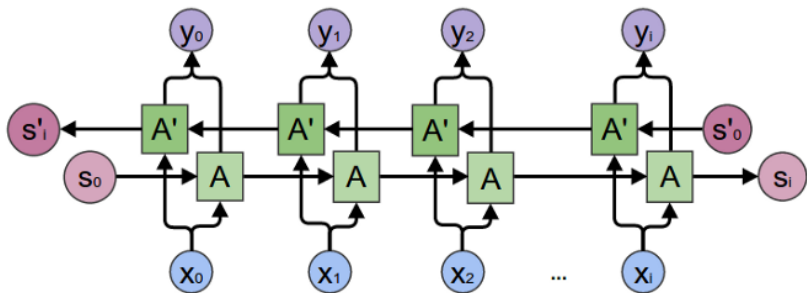
Loung vs Bahdanau

비교 항목	Bahdanau Attention	Luong Attention
Score 계산 방식	Additive Attention (선형 변환 후 비선형 함수)	Multiplicative Attention (점곱 방식)
Query	이전 디코더 hidden state (s_{j-1})	현재 디코더 hidden state (s_j)
Key	인코더 hidden states (h_i)	인코더 hidden states (h_i)
Value	인코더 hidden states (h_i)	인코더 hidden states (h_i)
Attention 시점	디코더 이전 hidden state 기반	디코더 현재 hidden state 기반
인코더 구조	Bidirectional RNN	단방향 RNN
계산 복잡도	상대적으로 높음	상대적으로 낮음
학습 효율성	더 정교한 Alignment 학습 가능	계산이 빠르고 효율적

Bidirectional RNN



Bidirectional RNN



Bidirectional RNN의 구조

1. 정방향 RNN:

- 입력 데이터를 시간 순서대로 (왼쪽 → 오른쪽) 처리합니다.
- 정방향 RNN이 계산하는 hidden state:

$$h_t^{\text{forward}} = f(W_{\text{forward}}x_t + U_{\text{forward}}h_{t-1}^{\text{forward}} + b)$$

2. 역방향 RNN:

- 입력 데이터를 반대 시간 순서 (오른쪽 → 왼쪽)로 처리합니다.
- 역방향 RNN이 계산하는 hidden state:

$$h_t^{\text{backward}} = f(W_{\text{backward}}x_t + U_{\text{backward}}h_{t+1}^{\text{backward}} + b)$$

3. 출력 결합:

- 정방향 hidden state (h_t^{forward})와 역방향 hidden state (h_t^{backward})를 결합하여 최종 hidden state를 생성합니다.

$$h_t = \text{concat}(h_t^{\text{forward}}, h_t^{\text{backward}})$$

- 결합 방법: Concatenation (벡터를 그대로 연결) 또는 Summation (두 벡터를 더함)

이해를 돕기 위한 예시

입력 시퀀스: x_0, x_1, x_2, x_3

- 정방향 RNN: $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow x_3$ (왼쪽 → 오른쪽으로 계산)
- 역방향 RNN: $x_3 \rightarrow x_2 \rightarrow x_1 \rightarrow x_0$ (오른쪽 → 왼쪽으로 계산)

각 시점 t 에서:

- 정방향 RNN: h_t^{forward}
- 역방향 RNN: h_t^{backward}

최종 hidden state는 두 값을 결합한 형태:

$$h_t = \text{concat}(h_t^{\text{forward}}, h_t^{\text{backward}})$$